

RippleMem: From Isolated Retrieval to Associative Recollection for Long-Term Agent Memory

Jingbo Ji¹ Lingyi Li² Xilong Cheng¹ Yuhao Zhou¹
Wenji Zhang¹ Yuting Tan¹ Yunxiao Qin^{1,3, *}

¹Communication University of China, Beijing, China.

²Zhilian Yinghe Technology Co., Ltd., Beijing, China

³State Key Laboratory of Media Convergence and Communication, Beijing, China
cocomilk23@mails.cuc.edu.cn, qinyunxiao@cuc.edu.cn

Abstract

LLM-based agents increasingly rely on external memory to support long-horizon reasoning and interaction. However, the main bottleneck is not simply storing past experience, but recovering the right set of evidence when relevant information is distributed across many interactions. Existing approaches struggle with this access problem. Full-context methods require noisy long-context search, flat retrieval often returns isolated and incomplete records, and graph-based memory systems can be expensive to construct while compressing rich event context. We introduce **RippleMem**, a long-term memory system that replaces one-shot retrieval with adaptive associative recollection. Inspired by cue-dependent episodic retrieval and associative completion, RippleMem stores interaction history as cue-rich episodic memory units and organizes them in an event-centric memory graph. Given a query, it first recalls relevant memory anchors through hybrid cues, then expands from these anchors along semantic and structural associations to recover missing supporting evidence. In this way, initially recalled memories serve not only as answer context, but also as cues for completing the evidence needed to answer. Experiments on **LoCoMo** and **LongMemEval-S** show that RippleMem achieves the best overall performance across evaluated settings, improving LLM-as-a-Judge accuracy by **3.95%** on LoCoMo and up to **11.87%** on LongMemEval-S, while reducing graph construction cost by about **30×**.

an agent often needs to preserve useful information from past interactions and recover it as evidence when later queries depend on earlier events, preferences, or decisions (Hatalis et al., 2024; Zhang et al., 2025). In long-horizon interaction, however, the information needed to answer a query is often not contained in a single past utterance or memory record. It may be distributed across temporally distant sessions, mixed with routine dialogue, and become useful only when multiple fragments are recovered together. Thus, a central challenge for long-term memory is not merely storing past experience, but recollecting an answerable set of evidence from distributed traces.

Recent long-term memory systems have therefore explored ways to make stored interaction histories more accessible at query time. Some systems store past interactions as compact memory records and retrieve them with semantic or query-aware mechanisms, supporting efficient personalization over long histories (Zhong et al., 2024; Chhikara et al., 2025; Liu et al., 2026). Memory operating systems and lifecycle-based frameworks further maintain, update, and reorganize memories over time, providing a more stable substrate for long-horizon access (Li et al., 2025; Hu et al., 2026a). A complementary line introduces explicit structure or association, using graphs, temporal links, or recollection-style retrieval to let memory access move beyond independent record lookup (Xu et al., 2025b; Rasmussen et al., 2025; Zhang et al., 2026).

Despite these advances, memory access can still fail to assemble an answerable evidence set. Flat query-centered lookup may stop at the most directly matched record, while graph expansion without an explicit evidence need may traverse nearby but non-supporting memories. In such cases, the system can miss supporting evidence that has been stored but is not recovered together with the initially surfaced memory.

Such failures raise a central question: when an

1 Introduction

Large language models (LLMs) are increasingly deployed as agents that must reason, plan, and interact over long horizons (Park et al., 2023; Shinn et al., 2023; Yao et al., 2023). In such settings, memory becomes an increasingly important capability rather than merely an auxiliary component, since

*Corresponding author

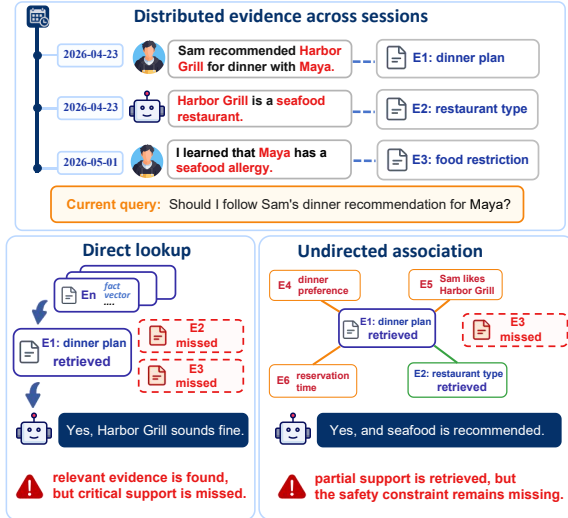


Figure 1: Failure modes in long-term memory access. Answer-critical evidence may be stored across sessions, while direct lookup or undirected association can still fail to assemble the missing support needed for an appropriate answer.

initially retrieved memory is relevant but incomplete, how should memory access continue to recover the missing support needed for answering? As illustrated in Figure 1, the query requires not only a dinner-plan memory, but also separately stored evidence about the guest’s food restriction and the restaurant type. Direct lookup may stop too early, while undirected association may surface related traces without ensuring that the missing evidence role is filled. Motivated by this observation, we argue that long-term memory access should be evidence-conditioned: recalled memories should serve as cues for recovering missing support, rather than as the endpoint of retrieval.

To operationalize this idea, we propose **RippleMem**, a long-term memory system that treats memory access as adaptive associative recollection. The name reflects the central mechanism: like ripples spreading from an initial point of contact, RippleMem starts from initially recalled memories and expands locally through linked episodic memories to recover missing evidence. This design is informed by cognitive accounts that organize experience into event-like units (Zacks et al., 2007; Baldassano et al., 2017) and view recall as cue-dependent and associative (Tulving and Thomson, 1973; Norman and O’Reilly, 2003). We use these accounts as design intuitions rather than mechanistic claims. RippleMem operationalizes them through a write-read design: the write phase stores interaction history as cue-rich episodic memory units linked by se-

mantic and structural associations, while the read phase uses initially recalled memories as cues for recovering additional support. This moves memory access beyond one-shot query matching and unguided graph traversal toward evidence completion over stored event memories.

In summary, our contributions are as follows:

- We formulate long-term memory access as an evidence recovery problem: relevant history may be stored but still fail to be recovered as an answerable evidence set when support is distributed across interactions. This framing highlights a limitation of one-shot query-memory matching.
- We propose **RippleMem**, an event-centric long-term memory system that combines cue-rich episodic memory construction with anchor-local associative recollection. Its key design lets recalled memories serve as both answer context and cues for recovering missing support through semantic and structural associations.
- We show that RippleMem achieves the best overall performance on LoCoMo and LongMemEval-S, relatively improving LLM-as-a-Judge accuracy by **3.95%** on LoCoMo and up to **11.87%** on LongMemEval-S over the strongest baselines, while reducing graph construction cost by about **30×** compared with graph-based memory baselines.

2 Related Work

2.1 Memory Mechanisms for LLM Agents

Long-context and retrieval-augmented memory. Large language models are constrained by finite context windows, and recent work improves long-context processing through extended context modeling and long-context prompting (Dai et al., 2019; Bai et al., 2024; Beltagy et al., 2020). However, longer context does not guarantee reliable memory access, as models remain vulnerable to evidence degradation and lost-in-the-middle effects (Liu et al., 2024). Retrieval-augmented generation externalizes memory through document or chunk retrieval (Lewis et al., 2021; Gao et al., 2024; Fan et al., 2024), but its effectiveness depends on whether the retriever surfaces evidence that may be distributed across turns, sessions, and time (Yu et al., 2024; Sorodoc et al., 2025). Graph-based

RAG further organizes external knowledge for relational access, including n-ary hypergraph representation in HyperGraphRAG, relational-path pruning in PathRAG, and graph foundation model-based reasoning in G-reasoner (Edge et al., 2025; Luo et al., 2025; Chen et al., 2026; Luo et al., 2026). This line primarily targets document- or knowledge-centric graphs, while RippleMem studies evidence recovery over evolving episodic interaction memory.

Long-term memory systems. A substantial body of work builds long-term memory for LLM agents by transforming interaction history into searchable memory entries (Lee et al., 2024; Tan et al., 2025b). MemGPT (Packer et al., 2024) treats memory as a virtual context space and manages information through memory paging. Memory-Bank (Zhong et al., 2024) stores user-specific information across interactions, while Mem0 (Chhikara et al., 2025) emphasizes concise long-term memory extraction for downstream retrieval. SimpleMem (Liu et al., 2026) further improves memory quality through semantic compression, structured indexing, and query-aware retrieval planning. Recent systems also organize memory with explicit structure. MemTree (Rezazadeh et al., 2025) supports coarse-to-fine access through hierarchical memory organization. Memory operating systems such as MemOS and EverMemOS (Li et al., 2025; Hu et al., 2026a) manage memory as a system-level resource. Graph-based architectures such as A-MEM (Xu et al., 2025b) and Zep (Rasmussen et al., 2025) link memories through structured associations or temporal knowledge-graph relations. While these approaches provide practical substrates for persistent memory, they leave open how memory access should continue when initially retrieved evidence is relevant but incomplete.

2.2 Episodic and Associative Memory Access

Cognitive accounts of episodic recollection. Human episodic memory offers a lens for long-term agent memory. Episodic memory concerns event-specific experiences and their contextual details (Tulving, 1972). Event segmentation accounts further suggest that continuous experience is organized into discrete event-like units, which support later understanding and recall (Zacks et al., 2007; Baldassano et al., 2017). Recollection also depends on binding item information with contextual features such as time, place, and surrounding

context (Davachi, 2006; Yonelinas et al., 2019). The encoding specificity principle suggests that retrieval succeeds when current cues overlap with information encoded during the original experience (Tulving and Thomson, 1973). Beyond direct cue matching, hippocampal indexing and pattern-completion accounts suggest that partial cues can reactivate associated traces and recover additional information (Norman and O’Reilly, 2003). RippleMem does not model human memory mechanistically, but adopts these ideas as design intuitions for event-level memory construction, cue-rich contextual binding, and associative recollection.

Associative access in memory systems. Recent memory systems explore structured access beyond one-shot matching. M-Flow (FlowElement AI, 2026) organizes memories into a cone-shaped multi-granularity graph and scores episode bundles through graph-routed path propagation. MemGAS (Xu et al., 2025a) constructs multi-granularity memory representations and selects among them for context construction. REMem (Shu et al., 2026) organizes time-aware gists and time-scoped facts in a hybrid graph and uses an agentic retriever for iterative evidence gathering. RF-Mem (Zhang et al., 2026) proposes a retrieval-side familiarity-recollection mechanism that adapts between direct retrieval and recollection-style expansion. These works show that long-term memory access benefits from structure, association, and adaptive retrieval. RippleMem is complementary to this line of work and focuses on how evidence already recovered during retrieval can serve as a cue for finding additional supporting memories.

3 Method

RippleMem is a long-term memory framework for recovering answer-supporting evidence from interaction history. It first constructs an event-centric memory substrate, and then uses this substrate to perform adaptive associative recollection at inference time.

3.1 Overview

Figure 2 illustrates the overall architecture of RippleMem, organized around a write-read design.

Write Phase. The write phase builds the memory substrate for later recollection. **Cue-Rich Episodic Memory Construction** converts dialogue history into self-contained event memories with semantic representations and episodic cues,

while the **Event-Centric Memory Graph** links these memories through semantic and structural associations. Together, they transform raw interaction history into a connected event memory space.

Read Phase. The read phase uses this event memory space to recover evidence for a query. **Adaptive Associative Recollection** first recalls candidate evidence and then, when needed, follows local associations from salient recalled memories to recover missing support. **Evidence Assembly** consolidates the resulting memories into a compact evidence context for grounded response generation.

Overall, RippleMem treats memory access as controlled evidence completion rather than one-shot retrieval, with recalled memories serving as both candidate evidence and cues for recovering additional support.

3.2 Cue-Rich Episodic Memory Construction

As illustrated in the write phase of Figure 2, RippleMem is motivated by event segmentation accounts (Zacks et al., 2007; Baldassano et al., 2017) and converts interaction history into cue-rich episodic memory units. Given a dialogue trace, we process it as contiguous turn-level windows $\{W_t\}$ with local overlap to preserve continuity across window boundaries. Each window is passed to a schema-guided LLM extractor, which returns a JSON list of memory units and may return an empty list when the window contains no durable event, preference, commitment, observation, or plan worth storing. The outputs are validated against the memory schema before being written to the memory bank.

Each memory unit follows a fixed schema:

$$m_i = (r_i, \mathbf{v}_i, P_i, L_i, T_i), \quad (1)$$

where r_i is a canonicalized event restatement that can be interpreted independently of the original dialogue window, $\mathbf{v}_i$ is the dense representation of r_i , and P_i , L_i , and T_i denote grounded participants, locations, and temporal cues or intervals when available.

To retain the contextual bindings central to episodic recollection (Davachi, 2006; Yonelinas et al., 2019), the structured cue fields preserve who, where, and when information for later cue-based access and association. The extractor is instructed to resolve speaker-dependent references into explicit participant names, anchor relative time expressions to absolute intervals only when supported by the

dialogue timestamp, and split multi-event windows into independently retrievable memory units. If a participant, location, or time expression cannot be grounded, the corresponding field is left empty rather than inferred.

The dense vector $\mathbf{v}_i$ supports semantic access, while (P_i, L_i, T_i) expose episodic cues that can later participate in structured and cross-cue association. Thus, the write phase does not merely compress dialogue into shorter summaries; it constructs cue-addressable event memories intended to support associative recollection.

3.3 Event-Centric Memory Graph

After memory construction, RippleMem organizes the memory bank as a sparse weighted event graph $G = (\mathcal{M}, \mathcal{E})$. As shown in Figure 2, each node is a memory unit, and $\mathcal{E} = \mathcal{E}_{\text{sem}} \cup \mathcal{E}_{\text{str}}$ contains typed links retained after association scoring. Each retained edge carries its corresponding association score as the edge weight and serves as a potential recollection path between two memory units.

Semantic association. Semantic association captures meaning-level proximity between memory restatements. Since each memory unit has a dense representation $\mathbf{v}_i$, RippleMem computes

$$s_{\text{sem}}(i, j) = \cos(\mathbf{v}_i, \mathbf{v}_j). \quad (2)$$

This channel preserves conceptual continuity in the graph, enabling a recalled event to cue semantically related memories during recollection.

Structural association. Structural association captures shared grounded episodic cues. For a pair of memory units, RippleMem computes a cue-similarity vector $\mathbf{u}_{ij} = [u_{ij}^P, u_{ij}^L, u_{ij}^T]$. Here u_{ij}^P and u_{ij}^L are Jaccard overlaps over canonicalized participant and location fields. For temporal cues, RippleMem assigns higher compatibility to overlapping or nearby intervals:

$$u_{ij}^T = \begin{cases} 1, & T_i \cap T_j \neq \emptyset, \\ \exp(-\Delta(T_i, T_j)/\tau), & \text{otherwise,} \end{cases} \quad (3)$$

where $\Delta(T_i, T_j)$ is the gap between two non-overlapping intervals and τ is a fixed temporal decay scale.

RippleMem aggregates cue-level similarities into a structural compatibility score over comparable cue types:

$$s_{\text{str}}(i, j) = \frac{\sum_{x \in \{P, L, T\}} \beta_x \mathbb{I}_x u_{ij}^x}{\sum_{x \in \{P, L, T\}} \beta_x \mathbb{I}_x}, \quad (4)$$

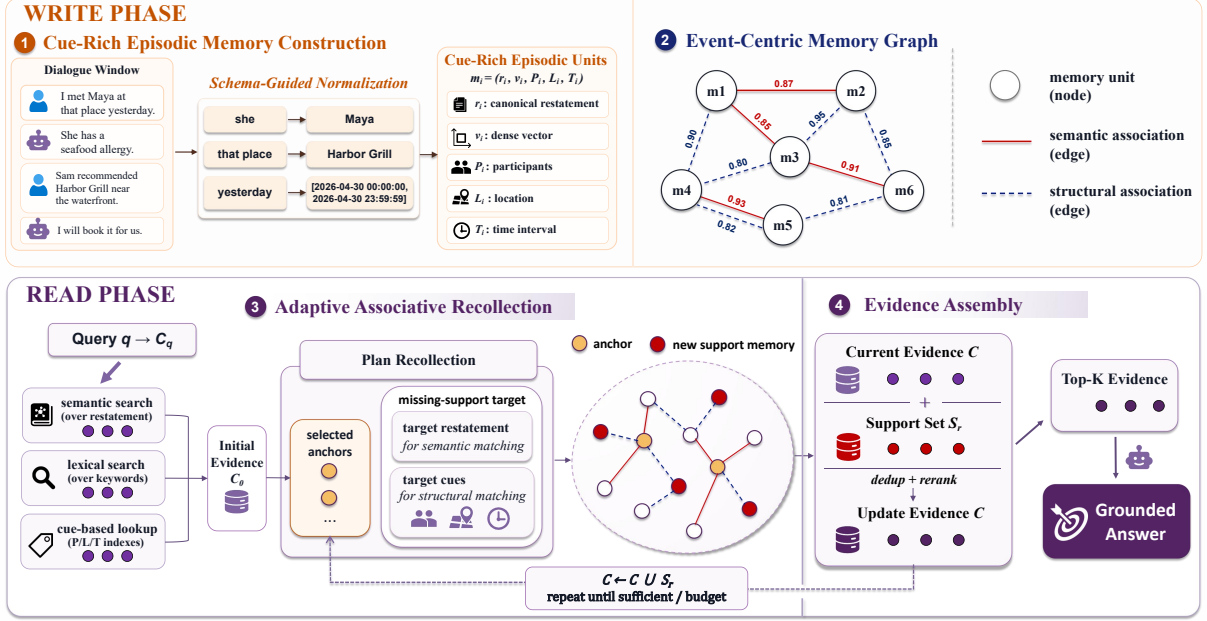


Figure 2: Illustration of the RippleMem framework, including cue-rich episodic memory construction, event-centric memory graph construction, adaptive associative recollection, and evidence assembly. In recollection, anchors define where to expand, while the missing-support target guides what support to recover.

where $\mathbb{I}_x$ indicates whether cue type x is present and comparable for both memory units, and β_x denotes a fixed cue weight. Pairs with no comparable cue type are not assigned a structural score.

Sparse graph construction. Graph construction is incremental and sparse. When a new memory unit is inserted, RippleMem obtains a bounded candidate pool from semantic nearest-neighbor search and cue-based indexes over participants, locations, and time. It then scores candidates through the two association channels. Semantic links are retained from candidates whose s_{sem} exceeds a score threshold, up to K_{sem} links; structural links are retained analogously using s_{str} , up to K_{str} links. A pair of memory units may be connected through either channel or both. During anchor-local recollection, the two typed channels can be traversed separately and then merged, allowing the system to recover memory units related by meaning, grounded episodic cues, or both.

3.4 Adaptive Associative Recollection

Hybrid initial recall. As shown in the read phase of Figure 2, RippleMem begins recollection by extracting retrieval cues c_q from a query q , including semantic, lexical, and grounded episodic cues. Consistent with the encoding-specificity view of retrieval (Tulving and Thomson, 1973), these cues

instantiate three complementary recall views:

$$C_0 = C_{\text{sem}} \cup C_{\text{lex}} \cup C_{\text{cue}}, \quad (5)$$

where C_{sem} , C_{lex} , and C_{cue} are obtained through semantic search, lexical matching, and structured cue matching, respectively. We initialize the current evidence state as $C \leftarrow C_0$ and update it after each recollection round.

Memory-anchor planning. Given the current evidence state C , RippleMem invokes a schema-constrained recollection controller Π_{rec} :

$$(d_r, A_r, g_r, s_r) = \Pi_{\text{rec}}(q, c_q, C). \quad (6)$$

The output specifies whether to continue (d_r), the selected anchors ($A_r \subseteq C$), the missing-support target (g_r), and an optional stop reason (s_r). Given the current evidence state C , the controller decides whether further recollection is needed. If so, it selects anchors that are central to the query or likely to evoke missing support through graph associations, and defines g_r using a target restatement and optional structured cues.

Anchor-local expansion. Given a valid plan, RippleMem expands only from the selected anchors within a bounded graph neighborhood:

$$U_r = \{m \in \mathcal{M} \setminus C \mid d_G(m, A_r) \leq h\}. \quad (7)$$

Here d_G is computed on the union graph induced by semantic and structural edges, while subsequent

Algorithm 1 Adaptive associative recollection

Require: query q , query cues c_q , memory graph G
Require: initial evidence C_0 , hop limit h , round budget R

```
1:  $C \leftarrow C_0$ 
2: for  $r = 1$  to  $R$  do
3:    $(d_r, A_r, g_r, s_r) \leftarrow \Pi_{\text{rec}}(q, c_q, C)$ 
4:   if  $d_r = \text{STOP}$  or  $A_r = \emptyset$  then
5:     break
6:   end if
7:    $U_r \leftarrow \text{LocalNeighbors}(G, A_r, h) \setminus C$ 
8:    $S_r \leftarrow \text{SelectSupport}(U_r, g_r, A_r)$ 
9:   if  $S_r = \emptyset$  then
10:    break
11:  end if
12:   $C \leftarrow C \cup S_r$ 
13: end for
14: return  $C$ 
```

selection retains channel-specific provenance. Expansion proceeds over the two graph channels. For each selected anchor, RippleMem collects candidates reachable within h hops through semantic edges and structural edges, excluding memories already in C . Candidates are then scored against the missing-support target: the target restatement is used for semantic matching, and the target cues are used for structural matching. The top candidates from the two channels are merged by memory identity to form the round-level support set S_r .

The evidence state is updated as $C \leftarrow C \cup S_r$. When candidates have similar target-match scores, RippleMem prefers candidates that are closer to the anchor and connected through stronger weighted paths. This loop continues until the evidence is sufficient, no new support is recovered, or the round budget is reached, as summarized in Algorithm 1.

3.5 Evidence Assembly

Evidence consolidation. As shown in the evidence assembly component of Figure 2, RippleMem converts the final evidence state C into a budgeted evidence context for response generation. It first consolidates memories by memory-unit identity. If the same memory unit is reached through multiple recall views, anchors, or anchor-local expansion paths, RippleMem keeps a single copy and merges its provenance information, producing a consolidated memory set $\bar{C}$.

RippleMem then applies a deterministic source-aware ordering to $\bar{C}$. The score combines query-memory alignment, retrieval provenance, and anchor status:

$$\rho(m) = \lambda_q a(q, m) + \lambda_p \pi(m) + \lambda_a \mathbb{I}_{\text{anc}}(m), \quad (8)$$

where $a(q, m)$ is the normalized semantic align-

ment between the query and the memory restatement, $\pi(m)$ summarizes retrieval-source rank, source score, and expansion-path support, and $\mathbb{I}_{\text{anc}}(m)$ indicates whether m served as a memory anchor. All weights are fixed before evaluation; the exact instantiation is provided in Appendix A.5.

The final evidence context E_K consists of the top- K memory units under this ordering, where K is a memory-count budget fixed before evaluation and kept unchanged across benchmarks. The answer generator receives the original query together with E_K and produces the final response conditioned on the assembled memories.

4 Experiments

We evaluate RippleMem through main benchmark comparisons, ablation studies, and phase-level efficiency analysis. Detailed dataset statistics, implementation settings, and qualitative cases are provided in the Appendix.

4.1 Experimental Setup

Datasets. We evaluate RippleMem on LoCoMo (Maharana et al., 2024) and LongMemEval-S (Wu et al., 2025), two long-term conversational memory benchmarks covering ultra-long dialogues and full-history user-agent interactions (He et al., 2025; Tan et al., 2025a; Wang and Zhao, 2024; Chu et al., 2024). Detailed dataset statistics and question categories are provided in Appendix A.1. We additionally evaluate RippleMem on EverMemBench (Hu et al., 2026b), a dynamic multi-party and multi-group conversational memory benchmark, with the full protocol and results reported in Appendix B.2.

Baselines. We compare RippleMem with representative long-context and memory-augmented baselines. On LoCoMo, we include Full-Context, Mem0 and Mem0^g (Chhikara et al., 2025), Zep (Rasmussen et al., 2025), MemGAS (Xu et al., 2025a), M-Flow (FlowElement AI, 2026), RE-Mem (Shu et al., 2026), SimpleMem (Liu et al., 2026), and RF-Mem (Zhang et al., 2026). On LongMemEval-S, we additionally compare with reported results from LightMem (Fang et al., 2026), MemU (NevaMind AI, 2025), MemOS (Li et al., 2025), and EverMemOS (Hu et al., 2026a). Baseline hyperparameters follow the original papers, public implementations, or official recommendations when available; brief descriptions are provided in Appendix A.2.

Method	Multi-Hop			Temporal			Open Domain			Single Hop			Average		
	F1 ↑	B1 ↑	J ↑	F1 ↑	B1 ↑	J ↑	F1 ↑	B1 ↑	J ↑	F1 ↑	B1 ↑	J ↑	F1 ↑	B1 ↑	J ↑
Full-Context	32.06	23.63	61.70	44.39	33.23	50.78	21.60	17.88	53.13	50.72	44.88	81.57	44.17	36.83	69.74
Mem0	32.42	24.70	62.06	47.91	42.03	64.49	20.20	15.56	53.13	39.31	34.70	62.54	38.65	33.20	62.27
Mem0 ^g	34.43	25.91	68.44	51.41	44.47	64.80	22.68	16.93	57.29	41.84	36.81	66.59	41.28	35.17	65.97
Zep	30.64	22.89	66.31	49.05	37.97	70.72	22.55	18.60	60.42	49.08	42.87	84.30	43.98	36.68	76.69
MemGAS	16.70	13.09	64.89	15.63	10.01	48.91	12.15	9.04	56.25	23.10	13.92	71.58	19.69	12.65	64.68
M-Flow	33.76	26.39	72.34	42.83	36.31	61.68	15.51	12.10	61.46	40.48	31.69	83.74	38.18	30.46	75.67
REMem	26.28	20.09	71.99	35.77	24.63	82.55	23.69	19.60	61.46	37.54	28.45	83.47	34.24	25.57	79.81
SimpleMem	37.12	29.98	78.01	56.92	42.77	76.01	24.75	19.77	63.54	55.45	49.21	89.42	50.48	42.51	82.92
RF-Mem [†]	37.04	29.00	75.89	59.62	45.15	80.69	26.54	21.87	68.75	54.14	48.00	89.42	50.43	42.30	83.83
RippleMem	38.58	31.12	77.67	62.37	47.29	85.67	25.71	20.93	70.83	56.44	49.79	92.75	52.49	44.05	87.14

Table 1: Main LoCoMo results. We report F1, BLEU-1 (B1), and LLM-as-a-Judge accuracy (J). RF-Mem[†] uses RippleMem’s extracted memory units with a matched per-question evidence budget. Bold denotes the best result.

Metrics. For LoCoMo, we report F1, BLEU-1, and LLM-as-a-Judge accuracy. F1 and BLEU-1 measure lexical overlap with reference answers, while the judge score evaluates semantic correctness beyond exact surface matching. For LongMemEval-S, we report binary LLM-as-a-Judge accuracy following its accuracy-style evaluation protocol. Appendix A.3 and Appendix A.4 provide metric aggregation and judge details.

Implementation Details. Unless otherwise specified, GPT-4.1-mini is the backbone LLM for memory extraction, query analysis, recollection planning, and answer generation, with all decoding temperatures set to zero. For LoCoMo, GPT-4.1-mini also serves as the judge, and Qwen3-Embedding-0.6B is used as the dense encoder. For LongMemEval-S, we follow two prior-aligned settings: SimpleMem-aligned with GPT-4.1-mini judge and Qwen3-Embedding-0.6B encoder, and EverMemOS-aligned with GPT-4o-mini judge and Qwen3-Embedding-4B encoder, taking baseline results from the corresponding prior work.

RippleMem uses fixed benchmark-specific budgets for memory construction, first-hop recall, graph expansion, and evidence assembly, as reported in Appendix A.5. For RF-Mem, we use the memory units extracted by RippleMem and match RippleMem’s per-question evidence budget to isolate the memory-access strategy.

4.2 Main Results

Tables 1 and 2 summarize the main results for LoCoMo and LongMemEval-S. We highlight three key observations.

Consistent overall gains across benchmarks. RippleMem achieves the best overall performance on both benchmarks. On LoCoMo, it obtains

52.49% F1, 44.05% BLEU-1, and 87.14% LLM-as-a-Judge accuracy. Against the strongest baseline for each metric, RippleMem relatively improves F1 and BLEU-1 over SimpleMem by 3.98% and 3.62%, respectively, and judge accuracy over RF-Mem by 3.95%. On LongMemEval-S, RippleMem achieves the best overall accuracy in both comparison groups, reaching 84.80% under the SimpleMem evaluation setting and 86.60% under the EverMemOS evaluation setting. These results show that RippleMem provides stable gains under both dialogue-level and full-history long-term memory evaluation settings.

Larger gains on evidence-distributed questions.

The improvements are especially clear on question types that require connecting dispersed memories. On LoCoMo temporal questions, RippleMem improves over SimpleMem by 5.45 F1 points and 9.66 judge-accuracy points. On open-domain questions, RippleMem achieves the highest judge accuracy, improving over RF-Mem from 68.75 to 70.83, although RF-Mem obtains higher lexical overlap. LongMemEval-S shows a similar pattern across the two comparison groups: RippleMem improves multi-session reasoning over SimpleMem (78.20 vs. 60.92) and EverMemOS (80.45 vs. 73.68), and remains competitive on temporal and knowledge-update questions where answers often depend on resolving event order or changed user states. This pattern is consistent with RippleMem’s design because expanding from recalled anchors helps recover supporting memories that are not directly matched by the original query.

Grounded recollection matters. Association- and recollection-style baselines provide competitive memory access signals. RF-Mem achieves the strongest baseline judge score on LoCoMo un-

Setting	Method	SS-User	Multi-S	SS-Pref	Temp. Reas	Know. Upd	SS-Asst	Overall
<i>SimpleMem setting</i>	Full-Context	47.14	30.08	60.00	27.06	41.03	32.14	35.40
	Mem0	87.14	50.37	63.33	40.60	69.23	48.21	58.40
	LightMem	88.57	47.37	76.67	85.71	92.30	21.43	69.20
	SimpleMem	85.71	60.92	76.67	83.46	79.48	75.00	75.80
	RippleMem	97.14	78.20	96.67	76.70	91.03	89.29	84.80
<i>EverMemOS setting</i>	MemU	67.14	42.10	77.67	17.29	41.02	19.64	38.40
	Zep	92.90	47.40	53.30	54.10	74.40	75.00	63.80
	Mem0	82.86	63.15	90.00	72.18	66.67	26.78	66.40
	MemOS	95.71	70.67	96.67	77.44	74.26	67.86	77.80
	EverMemOS	97.14	73.68	93.33	77.44	89.74	85.71	83.00
	RippleMem	95.71	80.45	83.33	84.21	88.46	94.64	86.60

Table 2: Main results on LongMemEval-S. We report LLM-as-a-Judge accuracy (%). SS denotes single-session; Asst and Pref denote assistant and preference; Multi-S denotes multi-session; Know. Upd and Temp. Reas denote knowledge-update and temporal-reasoning questions. Bold denotes the best result within each comparison group.

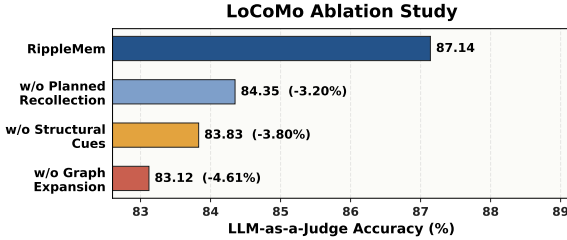


Figure 3: Ablation results on LoCoMo using LLM-as-a-Judge accuracy. Values in parentheses denote relative decreases compared with the full RippleMem model.

der matched memory units and evidence budgets, while REMem, MemGAS, and M-Flow introduce agentic, multi-granularity, or graph-routed retrieval, respectively. Nevertheless, RippleMem achieves the best overall judge accuracy and the strongest results in the temporal, open-domain, and single-hop categories. This suggests that the gain comes not merely from adding association or iterative retrieval, but from grounding recollection in stored event memories and using already recalled evidence to recover additional support. Appendix B.6 further evaluates this behavior on adversarial no-support questions, where the dialogue provides no valid supporting evidence.

4.3 Ablation Study

We conduct ablations on LoCoMo to isolate the contributions of structural cues, anchor-local graph expansion, and planned recollection. The backbone model, memory extraction, and answer generation are kept fixed; only the corresponding memory-access component is varied. Full ablation results with F1, BLEU-1, and judge accuracy are provided in Appendix B.3. A separate controlled replace-

Method	Build (s) ↓	Build Tok. ↓	Ans. Ctx. ↓	J ↑
Mem0 ^g	3623.63	4,243,278	628.17	65.97
Zep	3532.03	6,037,130	1,629.50	76.69
RippleMem	117.51	87,097	1,471.93	87.14

Table 3: Phase-level cost comparison on LoCoMo. Build time and build tokens are averaged per dialogue; answer-context tokens are averaged per question. J denotes overall LLM-as-a-Judge accuracy.

ment study of the edge-construction mechanism is reported in Appendix B.4.

Figure 3 shows that removing graph expansion causes the largest drop in judge accuracy, from 87.14 to 83.12, indicating that first-hop recall often misses supporting evidence. Removing structural cues lowers accuracy to 83.83, especially when related memories use different wording but share participants, locations, or temporal cues. Planned recollection also helps: without it, accuracy drops to 84.35 despite using the same expansion budget.

4.4 Phase-Level Efficiency Analysis

Table 3 decomposes efficiency into offline memory construction and answer-time evidence context. Compared with Mem0^g and Zep, RippleMem reduces construction time by approximately 30× and construction-stage tokens by 48.7–69.3×. Meanwhile, it maintains a relatively compact average answer-context budget of 1,471.93 tokens per question and achieves the strongest overall judge accuracy. These results demonstrate a favorable phase-level trade-off between persistent-memory construction, answer-time context size, and downstream effectiveness. The additional cost of query-time recollection control is analyzed separately in

5 Conclusion

In this paper, we introduced RippleMem, a long-term memory system for LLM agents that shifts memory access from isolated retrieval to adaptive associative recollection. Informed by cognitive views of event-based memory organization and cue-dependent recollection, RippleMem represents interaction history as cue-rich episodic memory units, links them in an event-centric memory graph, and recovers missing support through memory-anchor-based recollection. Experiments on LoCoMo and LongMemEval-S show strong overall performance, with particularly clear gains on evidence-distributed questions. These results suggest that reliable long-term memory depends not only on storing past experience, but also on organizing it so that partial recall can guide further evidence recovery. More broadly, recollection-aware memory access offers a promising direction for building more reliable, efficient, and context-sensitive LLM agents.

Limitations

RippleMem is evaluated primarily on text-only long-term conversational memory benchmarks. This setting does not cover multimodal interaction, embodied agents, or tool-use environments, where memory may involve visual observations, actions, and external tool states. Extending associative recollection to such settings remains an important direction for future work.

RippleMem uses LLM-mediated operations for memory extraction, query analysis, and recollection planning. While these operations enable flexible memory construction and adaptive recollection, they add latency and cost compared with single-pass retrieval baselines. Some components can be cached, batched, or executed asynchronously, but further improving end-to-end efficiency remains useful for large-scale deployment.

Finally, current benchmarks provide limited stress tests for continuously growing personal memory over very long timelines. Future evaluations with longer histories, evolving user states, memory aging, and privacy-preserving deletion would provide a more complete assessment of long-term agent memory systems.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (No. 62206259).

References

- Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. [LongBench: A bilingual, multi-task benchmark for long context understanding](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3119–3137, Bangkok, Thailand. Association for Computational Linguistics.
- Christopher Baldassano, Janice Chen, Asieh Zadbood, Jonathan W Pillow, Uri Hasson, and Kenneth A Norman. 2017. Discovering event structure in continuous narrative perception and memory. *Neuron*, 95(3):709–721.
- Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. [Longformer: The long-document transformer](#). *Preprint*, arXiv:2004.05150.
- Boyu Chen, Zirui Guo, Zidan Yang, Yuluo Chen, Junze Chen, Zhenghao Liu, Chuan Shi, and Cheng Yang. 2026. [PathRAG: Pruning graph-based retrieval augmented generation with relational paths](#). *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(36):30183–30191.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. [Mem0: Building production-ready ai agents with scalable long-term memory](#). *Preprint*, arXiv:2504.19413.
- Zheng Chu, Jingchang Chen, Qianglong Chen, Wei Jiang Yu, Haotian Wang, Ming Liu, and Bing Qin. 2024. [TimeBench: A comprehensive evaluation of temporal reasoning abilities in large language models](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1204–1228, Bangkok, Thailand. Association for Computational Linguistics.
- Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc Le, and Ruslan Salakhutdinov. 2019. [Transformer-XL: Attentive language models beyond a fixed-length context](#). In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2978–2988, Florence, Italy. Association for Computational Linguistics.
- Lila Davachi. 2006. Item, context and relational episodic encoding in humans. *Current opinion in neurobiology*, 16(6):693–700.
- Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2025. [From local to global: A](#)

- graph rag approach to query-focused summarization. *Preprint*, arXiv:2404.16130.
- Wenqi Fan, Yujuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li. 2024. A survey on rag meeting llms: Towards retrieval-augmented large language models. In *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining*, pages 6491–6501.
- Jizhan Fang, Xinle Deng, Haoming Xu, Ziyang Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Huajun Chen, and Ningyu Zhang. 2026. *Lightmem: Lightweight and efficient memory-augmented generation*. *Preprint*, arXiv:2510.18866.
- FlowElement AI. 2026. M-flow. https://github.com/FlowElement-ai/m_flow. GitHub repository.
- Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2024. *Retrieval-augmented generation for large language models: A survey*. *Preprint*, arXiv:2312.10997.
- Kostas Hatalis, Despina Christou, Joshua Myers, Steven Jones, Keith Lambert, Adam Amos-Binks, Zohreh Dannenhauer, and Dustin Dannenhauer. 2024. *Memory matters: The need to improve long-term memory in llm-agents*. *Proceedings of the AAAI Symposium Series*, 2(1):277–280.
- Junqing He, Liang Zhu, Rui Wang, Xi Wang, Gholamreza Haffari, and Jiaying Zhang. 2025. *MADial-bench: Towards real-world evaluation of memory-augmented dialogue generation*. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 9902–9921, Albuquerque, New Mexico. Association for Computational Linguistics.
- Chuanrui Hu, Xingze Gao, Zuyi Zhou, Dannong Xu, Yi Bai, Xintong Li, Hui Zhang, Tong Li, Chong Zhang, Lidong Bing, and Yafeng Deng. 2026a. *Evermemos: A self-organizing memory operating system for structured long-horizon reasoning*. *Preprint*, arXiv:2601.02163.
- Chuanrui Hu, Tong Li, Xingze Gao, Hongda Chen, Yi Bai, Dannong Xu, Tianwei Lin, Xiaohong Li, Yunyun Han, Jian Pei, and Yafeng Deng. 2026b. *Evaluating long-horizon memory for multi-party collaborative dialogues*. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining*.
- Kuang-Huei Lee, Xinyun Chen, Hiroki Furuta, John Canny, and Ian Fischer. 2024. *A human-inspired reading agent with gist memory of very long contexts*. *Preprint*, arXiv:2402.09727.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich K ttler, Mike Lewis, Wen tau Yih, Tim Rockt schel, Sebastian Riedel, and Douwe Kiela. 2021. *Retrieval-augmented generation for knowledge-intensive nlp tasks*. *Preprint*, arXiv:2005.11401.
- Zhiyu Li, Chenyang Xi, Chunyu Li, Ding Chen, Boyu Chen, Shichao Song, Simin Niu, Hanyu Wang, Jiawei Yang, Chen Tang, Qingchen Yu, Jihao Zhao, Yezhaohui Wang, Peng Liu, Zehao Lin, Pengyuan Wang, Jiahao Huo, Tianyi Chen, Kai Chen, and 20 others. 2025. *Memos: A memory os for ai system*. *Preprint*, arXiv:2507.03724.
- Jiaqi Liu, Yaofeng Su, Peng Xia, Siwei Han, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. 2026. *Simplemem: Efficient lifelong memory for llm agents*. *Preprint*, arXiv:2601.02553.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. *Lost in the middle: How language models use long contexts*. *Transactions of the Association for Computational Linguistics*, 12:157–173.
- Haoran Luo, Haihong E, Guanting Chen, Yandan Zheng, Xiaobao Wu, Yikai Guo, Qika Lin, Yu Feng, Zemin Kuang, Meina Song, Yifan Zhu, and Anh Tuan Luu. 2025. *HyperGraphRAG: Retrieval-augmented generation via hypergraph-structured knowledge representation*. In *Advances in Neural Information Processing Systems*, volume 38, pages 152206–152234.
- Linhao Luo, Zicheng Zhao, Junnan Liu, Zhangchi Qiu, Junnan Dong, Serge Panev, Chen Gong, Thuy-Trang Vu, Gholamreza Haffari, Dinh Phung, Alan Wee-Chung Liew, and Shirui Pan. 2026. *G-reasoner: Foundation models for unified reasoning over graph-structured knowledge*. In *The Fourteenth International Conference on Learning Representations*.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. *Evaluating very long-term conversational memory of LLM agents*. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870, Bangkok, Thailand. Association for Computational Linguistics.
- MemPalace Contributors. 2026. MemPalace. <https://github.com/MemPalace/mempalace>. GitHub repository.
- NevaMind AI. 2025. Memu. <https://github.com/NevaMind-AI/memu>. GitHub repository.
- Kenneth A Norman and Randall C O’Reilly. 2003. Modeling hippocampal and neocortical contributions to recognition memory: a complementary-learning-systems approach. *Psychological review*, 110(4):611.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2024. *Memgpt: Towards llms as operating systems*. *Preprint*, arXiv:2310.08560.

- Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22.
- Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. *Zep: A temporal knowledge graph architecture for agent memory*. Preprint, arXiv:2501.13956.
- Alireza Rezazadeh, Zichao Li, Wei Wei, and Yujia Bao. 2025. *From isolated conversations to hierarchical schemas: Dynamic tree memory representation for llms*. In *International Conference on Learning Representations*, volume 2025, pages 990–1023.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. *Reflexion: language agents with verbal reinforcement learning*. In *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652. Curran Associates, Inc.
- Yiheng Shu, Saisri Padmaja Jonnalagedda, Xiang Gao, Bernal Jiménez Gutiérrez, Weijian Qi, Kamalika Das, Huan Sun, and Yu Su. 2026. *REMem: Reasoning with episodic memory in language agent*. In *The Fourteenth International Conference on Learning Representations*.
- Ionut Teodor Sorodoc, Leonardo F. R. Ribeiro, Rexhina Blloshmi, Christopher Davis, and Adrià de Gispert. 2025. *GaRAGE: A benchmark with grounding annotations for RAG evaluation*. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 17030–17049, Vienna, Austria. Association for Computational Linguistics.
- Haoran Tan, Zeyu Zhang, Chen Ma, Xu Chen, Quanyu Dai, and Zhenhua Dong. 2025a. *MemBench: Towards more comprehensive evaluation on the memory of LLM-based agents*. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 19336–19352, Vienna, Austria. Association for Computational Linguistics.
- Zhen Tan, Jun Yan, I-Hung Hsu, Rujun Han, Zifeng Wang, Long Le, Yiwen Song, Yanfei Chen, Hamid Palangi, George Lee, Anand Rajan Iyer, Tianlong Chen, Huan Liu, Chen-Yu Lee, and Tomas Pfister. 2025b. *In prospect and retrospect: Reflective memory management for long-term personalized dialogue agents*. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8416–8439, Vienna, Austria. Association for Computational Linguistics.
- Endel Tulving. 1972. Episodic and semantic memory. *Organization of memory*, 1(381-403):1.
- Endel Tulving and Donald M Thomson. 1973. Encoding specificity and retrieval processes in episodic memory. *Psychological review*, 80(5):352.
- Yuqing Wang and Yun Zhao. 2024. *TRAM: Benchmarking temporal reasoning for large language models*. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 6389–6415, Bangkok, Thailand. Association for Computational Linguistics.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025. *Longmemeval: Benchmarking chat assistants on long-term interactive memory*. In *The Thirteenth International Conference on Learning Representations*.
- Derong Xu, Yi Wen, Pengyue Jia, Yingyi Zhang, wenlin zhang, Yichao Wang, Huifeng Guo, Ruiming Tang, Xiangyu Zhao, Enhong Chen, and Tong Xu. 2025a. *From single to multi-granularity: Toward long-term memory association and selection of conversational agents*. Preprint, arXiv:2505.19549.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025b. *A-mem: Agentic memory for llm agents*. Preprint, arXiv:2502.12110.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2023. *React: Synergizing reasoning and acting in language models*. In *The Eleventh International Conference on Learning Representations*.
- Andrew P Yonelinas, Charan Ranganath, Arne D Ekstrom, and Brian J Wiltgen. 2019. A contextual binding theory of episodic memory: systems consolidation reconsidered. *Nature Reviews Neuroscience*, 20(6):364–375.
- Wenhao Yu, Hongming Zhang, Xiaoman Pan, Peixin Cao, Kaixin Ma, Jian Li, Hongwei Wang, and Dong Yu. 2024. *Chain-of-note: Enhancing robustness in retrieval-augmented language models*. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 14672–14685, Miami, Florida, USA. Association for Computational Linguistics.
- Jeffrey M Zacks, Nicole K Speer, Khen M Swallow, Todd S Braver, and Jeremy R Reynolds. 2007. Event perception: a mind-brain perspective. *Psychological bulletin*, 133(2):273.
- Yingyi Zhang, Junyi Li, Wenlin Zhang, Pengyue Jia, Xianneng Li, Yichao Wang, Derong Xu, Yi Wen, Huifeng Guo, Yong Liu, and Xiangyu Zhao. 2026. *Evoking user memory: Personalizing llm via recollection-familiarity adaptive retrieval*. Preprint, arXiv:2603.09250.
- Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. 2025. A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems*, 43(6):1–47.
- Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. *Memorybank: Enhancing large language models with long-term memory*. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(17):19724–19731.

A Evaluation Protocol and Implementation Details

A.1 Evaluation Settings

For LoCoMo, we evaluate on 1,540 questions from 10 ultra-long dialogues, covering single-hop, multi-hop, temporal, and open-domain question answering. We report F1, BLEU-1, and LLM-as-a-Judge accuracy.

For LongMemEval-S, we evaluate on 500 questions over long user-agent interaction histories. The benchmark includes question types such as multi-session reasoning, temporal reasoning, knowledge update, user-specific facts, assistant-provided information, and preference recall.

For LongMemEval-S, we report two prior-aligned comparison settings to ensure comparability with previously reported baselines. In the SimpleMem-aligned setting, RippleMem uses GPT-4.1-mini as the LLM judge and Qwen3-Embedding-0.6B as the dense encoder. In the EverMemOS-aligned setting, RippleMem uses GPT-4o-mini as the LLM judge and Qwen3-Embedding-4B as the dense encoder. Baseline scores in each group are taken from the corresponding prior work, while RippleMem is evaluated under the same judge and encoder configuration.

A.2 Baseline Descriptions

Full-Context. This baseline directly provides the available interaction history to the LLM without using an external memory module, serving as a long-context prompting reference.

Mem0 and Mem0^g (Chhikara et al., 2025). Mem0 extracts compact long-term memories from interactions and retrieves them for downstream answering. Mem0^g augments this design with graph memory to capture relations among stored memories.

Zep (Rasmussen et al., 2025). Zep maintains a temporal knowledge graph over user interactions, supporting memory retrieval through structured entity and relation updates over time.

MemGAS (Xu et al., 2025a). MemGAS constructs memory associations across multiple granularities and selects relevant memory units for long-term conversational reasoning.

M-Flow (FlowElement AI, 2026). M-Flow organizes memory through a flow-style retrieval process, using structured memory routing to support multi-step access over stored interaction records.

REMem (Shu et al., 2026). REMem constructs a hybrid episodic memory graph from time-aware gists and time-scoped facts, and uses an agentic retriever with semantic, lexical, temporal, and graph-exploration tools for iterative evidence gathering.

SimpleMem (Liu et al., 2026). SimpleMem improves lifelong memory through semantic compression, structured indexing, and query-aware retrieval planning over compact memory entries.

RF-Mem (Zhang et al., 2026). RF-Mem proposes a familiarity-recollection retrieval mechanism that adapts between direct memory retrieval and recollection-style expansion. In our LoCoMo comparison, we evaluate its retrieval-side mechanism over the same extracted memory units as RippleMem and match the per-question evidence budget, isolating the effect of memory-access strategy. This setting does not reuse RippleMem’s graph expansion or recollection controller.

LightMem (Fang et al., 2026). LightMem focuses on lightweight memory management for efficient long-term memory use, reducing storage and retrieval overhead while maintaining user-specific information.

MemU (NevaMind AI, 2025). MemU is a memory management framework for LLM agents that maintains user memory through persistent storage and retrieval-oriented updates.

MemOS (Li et al., 2025). MemOS treats memory as a managed system resource and provides a unified framework for scheduling, storing, and retrieving different types of memory.

EverMemOS (Hu et al., 2026a). EverMemOS models memory as a lifecycle that transforms episodic traces into consolidated memory structures and uses reconstructive recollection for long-horizon reasoning.

A.3 Metric Aggregation

For LoCoMo, category-level scores are reported for diagnostic analysis, while average scores are computed over all evaluated questions rather than by macro-averaging category-level scores. For LongMemEval-S, overall accuracy is computed as the total number of correctly judged answers divided by the total number of questions.

A.4 LLM-as-a-Judge Protocol

We use an LLM-as-a-Judge protocol to evaluate semantic correctness when lexical overlap is insufficient for long-form memory answers. The judge receives only the question, the reference answer,

and the generated answer, and returns a binary correctness label. The same judge model is used for all methods within each evaluation setting, with temperature set to zero. The judge prompt is provided in Appendix E. For LoCoMo, GPT-4.1-mini is used as the judge. For LongMemEval-S, the judge follows the corresponding prior-aligned setting described in Appendix A.1.

A.5 Implementation Settings

Table 4 reports the benchmark-specific settings used by RippleMem. All values are fixed before evaluation and are not tuned per question or per category. The round budget R counts additional anchor-local recollection rounds after first-hop recall; we set $R = 1$ for all reported experiments.

Setting	LoCoMo	LongMemEval-S
Memory window	40 turns	10 turns
Window overlap	2 turns	0 turns
Semantic recall top- k	10	15
Lexical recall top- k	5	8
Structured recall top- k	5	8
Expansion hops	2	2
Max anchors	3	3
Semantic expansion top- k	5	5
Structural expansion top- k	5	5
Final evidence budget	30	30
Additional recollection rounds R	1	1

Table 4: Benchmark-specific implementation settings for RippleMem.

Table 5 reports the graph construction settings used by RippleMem. These values are also fixed before evaluation.

Setting	Value
Semantic edge candidate pool	20
Cue-based edge candidate pool	20
Max semantic edges per node	6
Max structural edges per node	6
Semantic edge threshold	0.85
Structural edge threshold	0.60
Participant cue weight	0.50
Location cue weight	0.20
Temporal cue weight	0.30
Temporal decay scale	7 days

Table 5: Graph construction settings used by RippleMem.

Source-aware ordering weights. The evidence ordering score uses fixed weights across all experiments. We set $\lambda_q = 1.25$, $\lambda_p = 1.0$, and $\lambda_a = 0.05$ for all experiments. The provenance term $\pi(m)$ aggregates retrieval-source contributions with fixed

Method	J $\uparrow$	Answer Ctx. $\downarrow$
MemPalace (Top-5)	81.75	3,802.01
MemPalace (Top-10)	86.69	7,502.59
RippleMem	87.14	1,471.93

Table 6: Aligned end-to-end comparison on LoCoMo Categories 1–4. J denotes LLM-as-a-Judge accuracy, and answer-context tokens are averaged per question. Top-10 is MemPalace’s documented LoCoMo configuration.

source weights: semantic search 1.0, lexical search 0.75, cue-based lookup 0.9, and anchor-local expansion 0.85. For each source, rank support is computed as $1/(5 + \text{rank})$ and combined with the available normalized source score. These weights are fixed before evaluation and are not tuned per dataset, category, or test question.

B Additional Experimental Results

B.1 Aligned Comparison with MemPalace

MemPalace (MemPalace Contributors, 2026) reports session-level retrieval recall, which is not directly comparable to end-to-end QA accuracy. We therefore conduct an aligned end-to-end comparison using the same LoCoMo split, Qwen3-Embedding-0.6B encoder, GPT-4.1-mini answer model and judge, and answer-level evaluation procedure. We retain MemPalace’s hybrid-v5, session-level retrieval without LLM reranking, and evaluate both Top-5 and Top-10 configurations.

Compared with MemPalace Top-5, RippleMem improves judge accuracy by 5.39 points while using approximately $2.6\times$ fewer answer-context tokens. Under MemPalace’s documented Top-10 configuration, RippleMem achieves slightly higher judge accuracy (87.14 versus 86.69) while using approximately $5.1\times$ fewer answer-context tokens, an 80.4% reduction. This result shows that RippleMem maintains higher end-to-end answer accuracy with substantially more compact evidence, rather than relying on a larger answer context.

B.2 Evaluation on EverMemBench

Benchmark. We additionally evaluate RippleMem on EverMemBench, using its EverMemBench-Dynamic release (Hu et al., 2026b). Unlike the dyadic conversations in LoCoMo and LongMemEval-S, EverMemBench evaluates long-term memory in multi-party, multi-group conversations with cross-group interactions, evolving information, and role-specific personas.

It contains five project histories with 51,023 dialogue turns and 2,400 questions, covering three dimensions and nine subtasks: fine-grained recall (single-hop, multi-hop, and temporal), memory awareness (constraint, proactivity, and update), and profile understanding (style, skill, and role).

Experimental setup. We compare RippleMem with Full Context, MemoBase, Mem0, Zep, MemOS, and RF-Mem. Full Context receives the complete dialogue history, following the EverMemBench setting. MemoBase, Mem0, Zep, and MemOS follow the implementations and memory/retrieval settings reported by the benchmark. RF-Mem follows the controlled setup used in our main experiments: it reuses RippleMem’s memory units and embeddings, with the number of returned memories matched to RippleMem for each question. RippleMem uses its event-centric memory graph construction and associative recollection pipeline.

All methods are evaluated on the same benchmark split using GPT-4.1-mini for answer generation and for judging open-ended questions, with the same scoring pipeline. Multiple-choice questions are evaluated by exact option matching. For RippleMem, we use BGE-M3 as the dense encoder, a memory window of 20 turns with one-turn overlap, semantic/lexical/structured retrieval budgets of 10/5/5, and a final evidence budget of 30 memories. These settings are fixed across all questions and subtasks. Overall accuracy is computed over all 2,400 questions rather than by macro-averaging the nine subtask scores.

Results. RippleMem achieves the highest overall accuracy of 54.75%, representing relative improvements of 28.13% over MemOS and 4.44% over the budget-matched RF-Mem baseline. It performs best on seven of the nine subtasks: single-hop, multi-hop, temporal, constraint, proactivity, update, and role. The gains therefore extend beyond factual retrieval to questions requiring cross-group evidence integration, evolving-memory resolution, and role-sensitive reasoning. Its improvement over RF-Mem under matched memory units and evidence budgets further indicates that evidence-conditioned associative recollection contributes beyond the use of recollection-style retrieval alone.

B.3 Full Ablation Results

Table 8 reports the complete LoCoMo ablation results with F1, BLEU-1, and LLM-as-a-Judge accu-

racy. The main paper visualizes judge accuracy for readability, while this table provides the full metric breakdown.

A small discrepancy appears between lexical-overlap metrics and judge accuracy. For example, w/o Structural Cues obtains higher F1/BLEU-1 in open-domain questions, but its judge accuracy is lower than the full model. This suggests that semantic-only expansion can sometimes retrieve memories with overlapping wording, while structural associations help organize grounded cues into evidence that better supports semantically correct answers.

B.4 Edge Construction Mechanism Ablation

We conduct a replacement ablation to examine whether RippleMem’s typed association scoring can be replaced by LLM-based edge decisions. We construct a controlled variant, denoted LLM-EDGE, in which only the edge-decision mechanism is replaced by GPT-4.1-mini, while all other components remain unchanged. The extracted memory units, candidate pool, per-node degree budget, recollection controller, graph expansion, answer generator, and judge remain unchanged. Both variants consider the same union of semantic top-20 and cue-based top-20 candidates. For each candidate pair, the verifier predicts SEM-SUPPORTS, STRUCT-SUPPORTS, BOTH, or NONE. Lexical matching remains a query-time recall channel and does not create persistent graph edges in either variant.

Table 9 reports the effectiveness comparison. The LLM-based verifier improves multi-hop judge accuracy from 77.67 to 79.79, indicating that flexible edge judgments can recover useful relations in some cases. However, it does not improve overall performance, obtaining 86.62 judge accuracy compared with 87.14 for RippleMem.

Table 10 compares the corresponding offline construction costs. LLM-EDGE requires approximately $27.7\times$ more construction tokens and $33.0\times$ more construction calls. RippleMem’s construction calls arise from memory extraction, whereas LLM-EDGE additionally invokes one batched candidate-edge verification call per source memory. Thus, although LLM-mediated edge decisions benefit particular question types, RippleMem’s typed associations provide a stronger overall accuracy–construction-cost trade-off.

Method	Fine-Grained Recall			Memory Awareness			Profile Understanding			Overall
	Single	Multi	Temp.	Const.	Proact.	Update	Style	Skill	Role	
Full Context	84.18	1.62	9.14	62.91	27.83	40.31	39.76	34.92	38.85	37.58
MemoBase	58.77	15.36	20.72	63.94	34.02	31.44	15.18	30.22	39.64	34.48
Mem0	56.08	9.11	4.56	67.03	55.74	50.92	24.11	30.77	35.48	37.23
Zep	71.62	10.47	15.38	66.43	45.18	44.92	28.64	34.81	45.12	40.16
MemOS	72.14	21.76	18.35	68.64	54.82	42.09	30.61	31.82	47.36	42.73
RF-Mem	91.55	15.66	21.00	77.36	67.21	54.10	30.68	39.64	49.49	52.42
RippleMem	92.02	22.09	21.33	78.11	71.66	58.96	31.25	36.69	53.06	54.75

Table 7: Results on EverMemBench. All values are accuracy percentages. Best results in each column are bolded. Overall accuracy is computed over all 2,400 questions.

Method	Multi-Hop			Temporal			Open Domain			Single Hop			Average		
	F1 ↑	B1 ↑	J ↑	F1 ↑	B1 ↑	J ↑	F1 ↑	B1 ↑	J ↑	F1 ↑	B1 ↑	J ↑	F1 ↑	B1 ↑	J ↑
w/o Structural Cues	37.24	28.90	74.82	59.96	45.85	81.93	29.98	24.79	62.50	55.37	49.17	90.01	51.43	43.24	83.83
w/o Graph Expansion	37.42	28.99	72.70	58.81	44.29	82.87	25.98	21.40	61.46	54.87	48.62	89.18	50.69	42.43	83.12
w/o Planned Recollection	36.36	28.90	77.66	60.37	45.95	81.31	26.39	22.28	60.42	55.20	48.94	90.49	51.03	42.99	84.35
RippleMem	38.58	31.12	77.67	62.37	47.29	85.67	25.71	20.93	70.83	56.44	49.79	92.75	52.49	44.05	87.14

Table 8: Full ablation results on LoCoMo. We report F1, BLEU-1 (B1), and LLM-as-a-Judge accuracy (J). Bold denotes the best result for each metric.

Method	Multi-Hop			Temporal			Open Domain			Single Hop			Average		
	F1 ↑	B1 ↑	J ↑	F1 ↑	B1 ↑	J ↑	F1 ↑	B1 ↑	J ↑	F1 ↑	B1 ↑	J ↑	F1 ↑	B1 ↑	J ↑
LLM-EDGE	37.81	30.65	79.79	61.67	46.69	83.80	30.31	24.66	70.83	55.64	48.98	91.80	52.05	43.63	86.62
RippleMem	38.58	31.12	77.67	62.37	47.29	85.67	25.71	20.93	70.83	56.44	49.79	92.75	52.49	44.05	87.14

Table 9: Effect of replacing RippleMem’s typed association scoring with GPT-4.1-mini edge decisions on LoCoMo. F1, B1, and J denote F1, BLEU-1, and LLM-as-a-Judge accuracy. Average scores are computed over all evaluated questions. Bold denotes the better result in each column.

Method	Build Tokens ↓	Build Calls ↓
LLM-EDGE	2,408,796	521.2
RippleMem	87,097	15.8

Table 10: Offline construction cost of the two edge-decision mechanisms on LoCoMo. Values are averaged per dialogue; lower is better.

B.5 Query-Time Recollection Controller Cost

Once query cues have been extracted, semantic, lexical, and cue-based lookup and bounded graph traversal require no additional LLM calls. RippleMem additionally invokes a schema-constrained recollection controller to assess evidence sufficiency, select memory anchors, and construct a missing-support target. On LoCoMo, this controller consumes an average of 2,880.6 prompt and completion tokens per question. Since the round budget is fixed to $R = 1$, the controller is invoked at most once after first-hop recall. The correspond-

ing ablation shows its utility: removing planned recollection under the same expansion budget reduces judge accuracy from 87.14 to 84.35.

B.6 Adversarial No-Support Analysis

LoCoMo Category 5 contains 446 adversarial questions for which the dialogue provides no valid answer support. Correct handling therefore requires recognizing insufficient evidence rather than producing an unsupported answer. RippleMem follows a conservative strategy: rather than immediately declaring that no support exists, the controller may invoke one additional bounded verification round ($R = 1$). Anchor-local expansion returns an empty support set when no candidate matches the missing-support target, so the evidence state is updated only when suitable support is found.

Table 11 shows that RippleMem achieves 86.32% judge accuracy on adversarial no-support questions. Removing the additional recollection round reduces accuracy to 84.75%, indicating that

Method	Category-5 J (%)
Zep	35.43
Mem0 ^g	38.12
SimpleMem	86.32
RF-Mem [†]	88.57
RippleMem w/o Round 2	84.75
RippleMem	86.32

Table 11: LLM-as-a-Judge accuracy on LoCoMo Category-5 adversarial no-support questions.

Metric	Categories 1–4	Category 5
Number of questions (N)	1540	446
Round-2 trigger rate (R2, %)	60.8	93.0
J(all)	87.1	86.3
J(R2)	81.7	87.5
J(stop)	95.5	71.0

Table 12: Recollection-controller behavior on answerable and adversarial no-support LoCoMo questions. R2 is the percentage of questions that trigger the additional bounded recollection round; J(all), J(R2), and J(stop) denote judge accuracy over all questions, triggered cases, and first-hop stopping cases, respectively.

bounded verification remains useful even when no valid answer support exists.

Table 12 further shows that the controller triggers an additional round for 93.0% of Category-5 questions, compared with 60.8% for Categories 1–4. Accuracy among the triggered adversarial cases reaches 87.5%. This behavior reflects RippleMem’s conservative response to insufficient first-hop evidence: it performs one bounded verification round but updates the evidence state only when suitable support is found.

C Reproducibility Artifacts

C.1 RippleMem Inference Procedure

Algorithm 2 summarizes the inference procedure of RippleMem. The algorithm abstracts away implementation-specific indexing details and highlights the controlled recollection loop used at test time.

D Qualitative Analysis

D.1 Case Studies

Figure 4 illustrates how RippleMem handles queries whose answers require evidence composition across sessions. The LoCoMo example requires recovering multiple purchase-related memories, while the LongMemEval-S example requires

Algorithm 2 RippleMem inference with adaptive associative recollection

Require: query q , memory graph $G = (\mathcal{M}, \mathcal{E}_{\text{sem}} \cup \mathcal{E}_{\text{str}})$, hop limit h , maximum rounds R , evidence budget K

- 1: $c_q \leftarrow \text{ExtractQueryCues}(q)$
- 2: $C_{\text{sem}} \leftarrow \text{SemanticRecall}(q, c_q, \mathcal{M})$
- 3: $C_{\text{lex}} \leftarrow \text{LexicalRecall}(q, c_q, \mathcal{M})$
- 4: $C_{\text{cue}} \leftarrow \text{StructuredCueRecall}(c_q, \mathcal{M})$
- 5: $C \leftarrow C_{\text{sem}} \cup C_{\text{lex}} \cup C_{\text{cue}}$
- 6: **for** $r = 1$ to R **do**
- 7: $(d_r, A_r, g_r, s_r) \leftarrow \Pi_{\text{rec}}(q, c_q, C)$
- 8: **if** $d_r = \text{STOP}$ or $A_r = \emptyset$ **then**
- 9: **break**
- 10: **end if**
- 11: $U_r^{\text{sem}} \leftarrow \text{Expand}(G_{\text{sem}}, A_r, h) \setminus C$
- 12: $U_r^{\text{str}} \leftarrow \text{Expand}(G_{\text{str}}, A_r, h) \setminus C$
- 13: $S_r^{\text{sem}} \leftarrow \text{SelectTop}(U_r^{\text{sem}}, g_r)$
- 14: $S_r^{\text{str}} \leftarrow \text{SelectTop}(U_r^{\text{str}}, g_r)$
- 15: $S_r \leftarrow \text{Dedup}(S_r^{\text{sem}} \cup S_r^{\text{str}})$
- 16: **if** $S_r = \emptyset$ **then**
- 17: **break**
- 18: **end if**
- 19: $C \leftarrow C \cup S_r$
- 20: **end for**
- 21: $\bar{C} \leftarrow \text{ConsolidateByMemoryId}(C)$
- 22: $E_K \leftarrow \text{TopK}_\rho(\bar{C}, K)$
- 23: **return** E_K

combining a total cost with a separately mentioned quantity. In both cases, the comparison setting answers from partial evidence, whereas RippleMem recovers the missing support needed to produce an answerable evidence context.

D.2 Worked Write-to-Read Trace

Table 13 presents a complete write-to-read replay that connects event-graph construction with query-time associative recollection. The example shows how typed associations established during the write phase enable RippleMem to recover recommendation evidence missing from first-hop recall.

E Prompt Templates

This section provides the core prompt templates used by RippleMem. The prompts are shown in their implementation-oriented form for reproducibility. For benchmarks with anonymous user/assistant roles, speaker-dependent references may be mapped to fixed canonical role names before cue extraction, so that role information can be represented in the participant field without changing the query meaning. Placeholders such as {query}, {dialogue_text}, and {current_evidence} are filled at runtime. Table 14 summarizes the purpose and output format of each prompt. The implementation field `lossless_restatement` corresponds to the mem-



Figure 4: Case studies on LoCoMo and LongMemEval-S. In both examples, the answer depends on evidence distributed across multiple conversation snippets. RippleMem recovers and composes the missing support, while the comparison setting produces an incomplete answer from partial evidence.

ory restatement r_i in Section 3.2.

Stage	Trace
Query	Which books has John recommended to James?
Reference	<i>The Name of the Wind</i> , <i>The Stormlight Archive</i> , <i>Kingkiller Chronicle</i> , and <i>The Expanse</i> .
Failure example	“ <i>The Name of the Wind</i> fantasy novel trilogy,” which omits the other three recommendations.
<i>Write phase: event-memory construction and association</i>	
Event memory m_1	Source D8:14. John recommended <i>The Name of the Wind</i> to James and praised its writing. Participants: {John, James}; location: $\emptyset$; time: [2022-04-29T14:36:00, 2022-04-29T14:36:00].
Event memory m_2	Source D14:9. James asked John which book series he loves and would recommend. Participants: {James, John}; location: $\emptyset$; time: [2022-06-16T17:07:00, 2022-06-16T17:07:00].
Event memory m_3	Source D14:10. John recommended <i>The Stormlight Archive</i> , <i>Kingkiller Chronicle</i> , and <i>The Expanse</i> . Participants: {John}; location: $\emptyset$; time: [2022-06-16T17:07:00, 2022-06-16T17:07:00].
Typed associations	RippleMem retains both semantic and structural associations between m_2 and m_3 . Their restatements concern the same recommendation exchange, providing semantic compatibility. The shared participant John and overlapping time intervals provide structural compatibility. Consequently, m_3 is reachable from m_2 through bounded local expansion.
<i>Read phase: adaptive associative recollection</i>	
First-hop evidence C_0	Hybrid recall retrieves m_1 , which supplies <i>The Name of the Wind</i> , and m_2 , which identifies the later recommendation exchange. It also retrieves a follow-up memory in which James thanks John for the recommendations and asks why they are his favorites. The remaining book-series names are absent from C_0 .
Recollection anchors A_r	The controller selects the recommendation-request memory m_2 and the recalled follow-up memory as anchors. These memories locate the relevant exchange but do not yet provide the missing recommendation names.
Target goal	Find additional books recommended by John to James.
Target restatement	Other books or book series recommended by John to James besides <i>The Name of the Wind</i> trilogy.
Target cues	Participants: {John, James}; locations: $\emptyset$. Time range: 2022-06-16T00:00:00 to 2022-06-16T23:59:59.
Anchor-local expansion	RippleMem expands from A_r over the retained semantic and structural associations. The target restatement guides semantic matching, while the participant and temporal cues guide structural matching. Candidates from both channels are merged by memory identity.
Recovered support S_r	The second recollection round recovers two supporting memories: (1) John suggested <i>The Expanse</i> for science-fiction fans and described it as epic; and (2) John recommended <i>The Stormlight Archive</i> and <i>Kingkiller Chronicle</i> as favorites, while also suggesting <i>The Expanse</i> . Together, these memories supply the three recommendation names missing from C_0 .
Evidence update	RippleMem updates the evidence state as $C \leftarrow C \cup S_r$. Evidence assembly then retains support for all four reference items in the final evidence context E_K .
Final answer	RippleMem answers: “ <i>The Name of the Wind</i> , <i>The Stormlight Archive</i> , <i>Kingkiller Chronicle</i> , and <i>The Expanse</i> series.”

Table 13: Worked write-to-read trace for associative recollection. The write phase constructs cue-rich event memories and typed associations; at inference time, first-hop memories identify a local recollection region from which RippleMem recovers recommendation evidence missing from the initial evidence state.

Prompt	Purpose	Output
Memory extraction	Construct cue-rich memory units	JSON memories
Query cue extraction	Parse first-hop retrieval cues	Query/cue JSON
Recollection planning	Select anchors and target	Plan JSON
Answer generation	Generate grounded response	Answer JSON
LLM-as-a-Judge	Evaluate correctness	Binary label

Table 14: Overview of prompt templates used by RippleMem.

Memory Extraction Prompt

Your task is to extract all valuable information from the following dialogues and convert them into structured memory entries.

Dialogue IDs in this window: {dialogue_ids}

[Current Window Dialogues]

{dialogue_text}

[Requirements]

1. Complete Coverage: Generate enough memory entries to ensure ALL valuable information in the dialogues is captured.
2. Force Disambiguation: Do not use pronouns such as he, she, it, they, this, or that.
3. Lossless Information: Each entry's lossless_restatement must be a complete, independent, understandable sentence.
4. Precise Extraction:
 - time_range: a pair [start, end] in ISO 8601 format if the memory can be grounded to an absolute time or interval from dialogue context; otherwise null.
 - If the memory refers to a specific event time, represent it as [t, t].
 - If the memory refers to a day, week, month, year, or other interval, represent the full interval.
 - If the dialogue only gives a vague time that cannot be grounded to an absolute interval, keep that wording in lossless_restatement and set time_range to null.
 - locations: all specific location names explicitly grounded in this memory.
 - persons: all explicit human personal names mentioned in this memory. Only include actual individual people.

[Output Format]

Return JSON only:

```
{
  "memories": [
    {
      "lossless_restatement": "Complete unambiguous restatement.",
      "time_range": ["ISO_START", "ISO_END"] or null,
      "locations": ["location1", "location2"],
      "persons": ["name1", "name2"]
    }
  ]
}
```

[Good Examples]

Example 1

Dialogues:

[2025-11-15T14:30:00] Alice: Bob, let's meet at Starbucks tomorrow at 2pm to discuss the new product

[2025-11-15T14:31:00] Bob: Okay, I'll prepare the materials

Output:

```
{
  "memories": [
    {
      "lossless_restatement": "Alice suggested at 2025-11-15T14:30:00 to meet with Bob at Starbucks on 2025-11-16T14:00:00 to discuss the new product.",
      "time_range": ["2025-11-16T14:00:00", "2025-11-16T14:00:00"],
      "locations": ["Starbucks"],
      "persons": ["Alice", "Bob"]
    },
    {
      "lossless_restatement": "Bob agreed to attend the meeting and committed to prepare relevant materials.",
      "time_range": null,
      "locations": [],
      "persons": ["Bob"]
    }
  ]
}
```

Now process the above dialogues. Return ONLY the JSON object, no other explanations.

First-Hop Query Cue Extraction Prompt

You are extracting retrieval cues for first-hop memory retrieval.

Given the question, produce a single JSON object with:

- original_query: the original question
- semantic_query: the full original query with only speaker references normalized; replace first-person references such as I, me, and my with Alice, and replace references to the assistant with Bob when applicable. Do not delete any content or information.
- keywords: 2-6 short lexical cues for BM25/fulltext retrieval
- persons: explicit named individual participants mentioned in the question
- locations: explicit concrete locations mentioned in the question
- time_range: a pair [start, end] in ISO 8601 format if the question contains a time reference that can be grounded to an absolute date or interval; otherwise null

Rules:

- semantic_query must preserve the original meaning exactly.
- Do not hallucinate missing people, places, or time.
- keywords should be short phrases, not full sentences.
- If no explicit person, location, or time appears, return empty lists or null.

Question: {query}

Recollection Planning Prompt

You are deciding whether to continue a graph-based memory recollection step.

Round: {round_name}
Question: {query}
Query analysis: {query_analysis}
Current evidence:
{current_evidence}

Goal:

- Expand only if the current evidence is still insufficient.
- If the current evidence already contains a direct answer or clearly sufficient support, do not expand.
- Treat the current evidence set as the memories that came to mind first.
- Select up to {max_expand_nodes} anchors only from the current evidence set.
- An anchor should be a strong starting point for further recollection: it should either be directly useful for answering the question, or serve as a good local gateway to the missing evidence.
- Do not select all relevant memories; select only the few best recollection starting points.
- Produce exactly one shared evidence_target for this round.
- The evidence_target should describe the missing memory to retrieve next, not the final answer and not a paraphrase of the original question.
- Write target_restatement in the style of a factual memory sentence, similar to a memory node's lossless_restatement, not as a question or retrieval instruction.
- Prefer a concrete missing-evidence description, for example by specifying the missing fact type, participant, location, or time cue when available.
- If time is relevant to the missing evidence, include a normalized ISO 8601 time interval in time_range; otherwise return null.
- Stop when the current evidence is already sufficient, or when further local expansion is unlikely to help.

Example:

Question: When did Caroline and Melanie go to a pride festival together?

Current evidence set:

- memory_a: Caroline attended a pride parade on 2023-08-11 and felt proud and supported.
- memory_b: Melanie said the pride parade looked awesome and asked Caroline if Caroline joined in.
- memory_c: Caroline and Melanie often support each other in LGBTQ-related activities.

A good response:

```
{
  "continue": true,
  "stop_reason": null,
  "anchors": ["memory_b"],
  "evidence_target": {
    "goal": "find_shared_pride_event_date",
    "target_restatement": "Caroline and Melanie attended a pride-related event together.",
    "persons": ["Caroline", "Melanie"],
    "locations": [],
    "time_range": null
  }
}
```

Return JSON only:

```
{
  "continue": true,
  "stop_reason": null,
  "anchors": ["memory_id_1", "memory_id_2"],
  "evidence_target": {
    "goal": "short missing-evidence label",
    "target_restatement": "A concise restatement of the missing evidence to retrieve next.",
    "persons": ["optional explicit people names"],
    "locations": ["optional explicit locations"],
    "time_range": ["ISO_START", "ISO_END"] or null
  }
}
```

Answer Generation Prompt

Answer the user's question based on the provided context.

User Question: {query}

Relevant Context:
{context_str}

Requirements:

1. First, think through the reasoning process.
2. Then provide a very concise answer.
3. Answer must be based only on the provided context.
4. All dates in the response must be formatted as DD Month YYYY when dates are required.
5. Return your response in JSON format.

Output Format:

```
{
  "reasoning": "Brief explanation of the reasoning process.",
  "answer": "Concise answer."
}
```

Example:

Question: "When will they meet?"

Context: "Alice suggested meeting Bob at 2025-11-16T14:00:00..."

Output:

```
{
  "reasoning": "The context explicitly states the meeting time as 2025-11-16T14:00:00.",
  "answer": "16 November 2025 at 2:00 PM"
}
```

Now answer the question. Return ONLY the JSON, no other text.

LLM-as-a-Judge Prompt

Your task is to label an answer to a question as CORRECT or WRONG.

You will be given:

- (1) a question,
- (2) a gold answer,
- (3) a generated answer.

The point of the question is to ask about something one user should know about the other user based on their prior conversations.

The gold answer will usually be concise and include the referenced topic. The generated answer may be longer. Mark the generated answer as CORRECT if it matches the same core information as the gold answer.

For time-related questions, accept equivalent date or time expressions when they refer to the same time period. Even if the format differs, consider it CORRECT if it refers to the same date or time.

Question: {question}

Gold answer: {gold_answer}

Generated answer: {generated_answer}

Return JSON only:

```
{
  "label": "CORRECT" or "WRONG"
}
```